

Calibração de Estimativa — Medições do Repositório SILE

Documento gerado para calibrar a estimativa de esforço de outro projeto comparando com o SILE. Tudo aqui foi medido no repositório real (git, contagem de arquivos versionados, `.planning/STATE.md`). Marcações: **[medido]** = extraído direto do repo · **[estimado]** = derivado de evidência · **[não medido]** = git não registra.

- **Repositório:** `/Users/filipefalcao/Downloads/fabrica/sile`
- **Data da medição:** 2026-06-19
- **Janela do projeto:** 2026-06-09 → 2026-06-19

Resumo executivo (TL;DR)

Dimensão	Valor medido
Janela	11 dias corridos, commit em todos os 11 (sprint 7 dias/semana)
Time	1 dev solo (1 autor git), assistido por IA em ondas paralelas
Commits	748 (média ~68/dia, pico 265 num domingo)
Código-fonte real	~130.700 linhas (das quais 47.304 são testes , ~36%)
Migrations / Models	69 / 48
Endpoints	162 (0 resource → ~162 reais)
Telas / Componentes	65 / 51
Testes / Asserts	~1.572 métodos / ~5.518 (grep) — ~7.358 em run completo logado
Fases (módulos)	15 planejadas, 13 concluídas (138 planos, 135 concluídos)
Docs de planejamento	338 arquivos / 57.406 linhas

Throughput (÷ 11 dias): ~6,3 tabelas/dia · ~14,7 endpoints/dia · ~5,9 telas/dia · ~143 testes/dia · ~1,2 módulo/dia (~6–8 módulos/semana).

Diferenças que tornam o projeto-alvo MAIS LENTO que este: multi-tenant (×1,25) + CPF cifrado/blind index (×1,15) + (se IA limitada/compartilhada) resets/retrabalho (×1,2–1,5). **Multiplicador combinado estimado: 1,4x (IA dedicada) a 1,9x (IA limitada); centro provável 1,5x–1,8x.**

⚠ **Não use linhas/dia (~12 mil) como baseline humano** — é artefato de IA. Calibre por endpoints/dia, telas/dia, testes/dia e módulos/semana.

1. Identidade do projeto

Domínio [medido]: SILE — Sistema de Licenciamento Eletrônico da SEDUR (Salvador/BA). Viabilidade locacional de atividade econômica de forma automática e auditável: motor de regras da LOUOS (Lei 9.148/2016), classificação de risco por CNAE (Decreto 32.636/2020), georreferenciamento (PostGIS/Leaflet/Nominatim), fluxo expresso (deferimento/indeferimento automático), análise técnica humana, comunicação multicanal, IA de apoio e relatórios.

Stack [medido]: Laravel 13 + Inertia v3 + React 19 + Tailwind 4, PHP 8.3+, PostgreSQL/PostGIS, Redis. Monólito modular por domínio (Services/Concerns/StateMachines, eventos de domínio + listeners auto-descobertos). Pacotes-chave: `spatie/permission` (RBAC), `spatie/activitylog` (auditoria), `firebase/php-jwt` + `socialite` (gov.br), `dompdf` / `openspout` (PDF/XLSX), `laravel/ai`.

Complexidade de negócio: ALTA [estimado, evidência forte]. Motor de regras legais versionado (Quadros LOUOS 7/10/11/11A), classificação de risco com condicionantes, consultas espaciais reais (ST_Intersects/ST_Contains), máquinas de estado, decisão automática com fundamentação legal e trilha de auditoria em toda decisão.

Requisito	Tem?	Detalhe medido
Multi-tenant	NÃO	<code>tenant / municipio_id / landlord / stancl</code> em <code>app/ database/ config/ routes/</code> = 0 ocorrências. Mono-município.
Segurança/LGPD	Parcial	Auditoria, painel LGPD (HU-102), <code>personal_data</code> , máscara de CPF (<code>cpf_masked</code>), detecção de abuso. MAS: CPF é <code>string(11) unique</code> em <code>texto plano</code> ; <code>User::casts()</code> não cifra CPF;cripto só para segredos (API keys, senha de e-mail). Sem blind index, sem cifra de campo de CPF.
Parametrização	SIM (forte)	90 parâmetros + 29 permissões; feature toggles (<code>features.ia_*</code> , <code>features.notificacao_whatsapp</code> , <code>features.deteccao_abuso</code>); HU-014 com histórico auditado e efeito sem deploy.
Auditoria	SIM (transversal)	<code>HasAuditoria</code> + <code>AuditService</code> + <code>access_logs</code> desde a Fase 1 (RN-002); <code>activitylog</code> com versão de regras.

Os dois itens mais caros da régua de segurança do projeto-alvo (multi-tenant e CPF cifrado + blind index) NÃO foram pagos aqui.

2. Janela de tempo [medido no git]

- 1º commit: 2026-06-09 (terça) · último: 2026-06-19 (sexta).
- Dias corridos: 11 · Dias com commit: 11 de 11 — sprint 7 dias/semana, incluindo sábado 13 (82) e domingo 14 (265, pico).

Distribuição de commits/dia:

09	ter	55		10	qua	77		11	qui	16		12	sex	48		13	sáb	82				
14	dom	265		15	seg	118		16	ter	40		17	qua	20		18	qui	15		19	sex	12

- Dias úteis efetivamente trabalhados: uso 11 dias (há commit em cada). Composição: 9 dias de semana + 2 de fim de semana.
- Time: 1 pessoa (autor git único Falcao, 748–749 commits). Solo, full-stack.
- Horas/dia: [não medido]. Indício de intensidade: ~68 commits/dia, pico 265 num domingo (padrão de orquestração de agentes de IA). Foco aparentemente exclusivo.

3. Condições de execução

- Assistido por IA? SIM, intensivo [estimado, evidência forte]. GSD + Superpowers + subagentes especialistas do Cursor (`.cursor/agents/`). `STATE.md` descreve execução em ondas paralelas ("12 planos em 6 waves", agentes concorrentes). Ferramenta/modelo exato: [não medido].
- Tokens dedicados ou compartilhados? Dedicados [estimado]. Débito (68 commits/dia, ondas concorrentes) é incompatível com tokens limitados. Retrabalho registrado é de coordenação (regressões de baseline, "race no índice git", working tree contaminado pela Fase 14), não de reset por limite de contexto.
- Starter kit / reuso externo [medido + estimado]: SIM — Laravel skeleton, Fortify (auth), `spatie/permission` (RBAC), `spatie/activitylog` (auditoria base), `dompdf / openspout` (export), template de UI (TailAdmin). Economia estimada: ~15–25% do esforço total.
- TDD real? SIM, estrito [medido]. 277 arquivos de teste, ~1.572 métodos, ciclo RED→GREEN com commits separados, guardião-entrega "APROVADO com evidência fresca", anti-fachada com degradação honesta. Testes = ~36% do código.
- Produção ou protótipo? Pré-produção/homologação [medido]. Núcleo real com testes reais, mas não foi à produção: Fase 13 (Integrações REDESIM/Regin/SEFAZ/GIS) bloqueada por terceiros; gov.br implementado e desligado aguardando

credenciamento. Única integração externa validada ponta a ponta: provedor de IA (chamada real, tokens 37/14).

4. Volume entregue [medido]

Item	Quantidade	Fonte
Migrations (tabelas)	69	database/migrations/*.php
Models	48	app/Models/**/*.php
Seeders / Factories	27 / 45	por diretório
Controllers	66	app/Http/Controllers/**
Rotas/endpoints	162	declarações Route::* (0 resource)
Páginas frontend	65	resources/js/pages/**/*.tsx
Componentes frontend	51 (123 .tsx no total)	resources/js/components/**/*.tsx
Arquivos de teste	277	258 Feature + 19 Unit
Métodos de teste	~1.572	grep public function test / #[Test] (cruza com ~1.554 executados no STATE)
Asserts	~5.518 (grep) · ~7.358 (run completo logado)	grep estático vs. log STATE
Commits	748	git rev-list --count HEAD
Linhas adicionadas (git, tudo)	+292.149 / -10.196	git log --numstat (inclui lock/build/docs)
Código-fonte real (atual)	~130.700	app 41.323 · resources/js 30.212 · tests 47.304 · database 8.873 · config 2.181 · routes 807
Docs de planejamento	338 arquivos / 57.406 linhas	.planning/**/*.md

As +292k linhas do git são infladas por composer.lock , package-lock.json , public/build/assets/* minificados e 338 docs. O número honesto de código é ~130.700 linhas (47.304 de testes).

5. Throughput normalizado

Denominador = 11 dias trabalhados. Entre parênteses, alternativa por 9 dias úteis (seg-sex).

Métrica	Conta	Por dia trabalhado	(por dia útil)
Tabelas/migrations	69 ÷ 11	6,3	(7,7)
Models	48 ÷ 11	4,4	(5,3)
Endpoints	162 ÷ 11	14,7	(18,0)
Telas/páginas	65 ÷ 11	5,9	(7,2)
Testes	1.572 ÷ 11	142,9	(174,7)
Asserts	5.518 ÷ 11	~502	(613)
Linhas de fonte	130.700 ÷ 11	~11.880	(14.520)

Módulos por semana. Módulo = **fase do ROADMAP** (contexto de domínio delimitado). 15 fases planejadas, **13 concluídas** (138 planos, 135 concluídos).

- $13 \div 11 = \sim 1,2$ **módulo/dia** → **~8,3 módulos/semana** (corrido) ou **~5,9/semana** (dias úteis).
- Granularidade fina: $135 \text{ planos} \div 11 = \sim 12,3$ **planos/dia**.

Linhas/dia (~12 mil) é artefato de IA. Para calibrar use endpoints/dia, telas/dia, testes/dia e módulos/semana.

6. Esforço por arquétipo de tarefa [estimado: git + estrutura de fases]

Arquétipo	Tempo médio real	Evidência
CRUD simples sobre base pronta (model + migration + 2 telas + validação)	~0,25–0,5 dia útil	Ferriados (HU-137) e Setores como <i>uma task</i> dentro de planos maiores no mesmo dia; CRUDs admin (cnaes/params/usuários/perfis) nos ~4 primeiros dias junto a muito mais.
Módulo de domínio novo (regra/máquina de estados + ~3 telas + testes)	~1–1,5 dia útil	Fases 5 (LOUOS), 6 (Risco), 9 (Expresso): ~1 fase/dia em 13→17/jun, acelerando pelo reuso interno.
Tela/página de formulário denso isolada	~0,3–0,6 dia	Wizard multi-etapas (Fase 8); 4 páginas React do plano 15-14 no mesmo dia.
Integração externa	REDESIM/SEFAZ/Regin/GIS = NÃO ENTREGUE [não medido — bloqueado por terceiros]. Única validada: provedor de IA (OpenAI-compatível) na fundação da Fase 14, ~1 dia com chamada real comprovada. gov.br construído mas desligado (não validado em homologação).	

7. O que acelerou e o que atrasou

Aceleradores (top 3):

1. **IA dedicada em ondas paralelas (subagentes)** — multiplicador dominante. 68 commits/dia (pico 265 num domingo), waves concorrentes. *Ganho estimado: 5–10x vs. 1 dev sem IA.*
2. **Skeleton + pacotes + template de UI** (Laravel/Fortify/spatie-permission/spatie-activitylog/dompdf/openspout/TailAdmin). Auth, RBAC, trilha e export não foram do zero. *Ganho: ~15–25% do esforço total.*
3. **Reuso interno agressivo** — serviços reaproveitados entre fases (ConsultaViabilidadeService → Fases 8/9; `rule_versions` da Fase 6 → Fase 5; listeners `ResultadoEmitido` → Fase 10; `ReportExporter` reusado por 7 telas). *Ganho: Fases 7–12 em ~3 dias.*

Gargalos/atrasos (top 3):

1. **Dependências externas (SEDUR/Regin/SEFAZ/GIS/gov.br)** — Fase 13 inteira não entregue + degradação honesta em ~4 fases. *Custo: 1 fase parada + retrabalho de degradação.*
2. **Testes dual-driver PostGIS (SQLite + postgres)** — bug de migrate pulado (fix `0cb1c8b`), migrations/seeder driver-aware, CI em 2 passos. *Custo: retrabalho recorrente no harness.*
3. **Coordenação de agentes paralelos + working tree contaminado** (Fase 14 não commitada) — regressões de baseline, staging seletivo manual. *Custo: overhead repetido de reconciliação/verificação.*

8. Veredito de comparação

Em uma frase: o ritmo foi possível porque foi **1 dono em sprint 7 dias/semana com IA dedicada de alto débito (ondas paralelas), sobre skeleton + pacotes prontos + forte reuso interno, num escopo single-tenant onde as integrações externas mais caras foram adiadas (bloqueadas), não resolvidas** — com TDD/anti-fachada **já embutidos** nestes números.

Fatores do projeto-alvo que o deixam mais lento que este:

Fator	Neste projeto	No alvo	Multiplicador
Multi-tenant (multi-município)	ausente (0 referências)	scoping transversal, isolamento, testes por tenant	×1,25
CPF cifrado + blind index	CPF texto plano + máscara	cifra de campo + índice cego + unicidade/busca em todo ponto de CPF	×1,15
TDD estrito + anti-fachada + parametrização	já feito aqui	igual	×1,0 (neutro) — custo já está no número
Tokens de IA compartilhados/limitados	dedicados (sem reset evidente)	se limitados → resets + retrabalho	×1,2–1,5 (maior fator; depende do setup)

Combinação:

- IA dedicada como aqui: $1,25 \times 1,15 \approx \sim 1,4x$ mais lento.
- IA limitada/compartilhada: $1,25 \times 1,15 \times 1,35 \approx \sim 1,9x$ mais lento.
- Faixa: 1,4x–1,9x · centro provável 1,5x–1,8x.**

Como recalcular a estimativa do alvo:

- Pegue o throughput daqui em unidades **não-infladas por IA** (endpoints/dia ≈ 15 , telas/dia ≈ 6 , módulos/semana $\approx 6\text{--}8$ corrido).
- Divida pelo multiplicador conforme as condições reais do alvo.
- Não conte integrações externas como capacidade** — aqui foram bloqueadas, não entregues; no alvo são trabalho real adicional não representado nestes números.
- Ajuste o denominador de dias para o regime do alvo (este foi 7 dias/semana; se o alvo for 5 dias/semana, a duração calendário cresce proporcionalmente).

Linha do tempo das fases [medido em STATE.md; datas iniciais inferidas pela distribuição de commits]

Período	Fases entregues
09–12/jun (ter–sex)	1 (Identidade/Acesso/Auditoria), 2 + 2.4 (Administração base), 3 (Cadastro empresarial), 3.1 (Fundação assíncrona), 3.2 (gov.br — construída e desligada)
13/jun (sáb)	4 (Georreferenciamento/PostGIS)
14/jun (dom)	5 (Motor LOUOS) + 6 (Classificação de risco)
14–15/jun	7 (Consulta prévia), 8 (Solicitação), 9 (Fluxo expresso), 10 (Análise técnica)
15/jun (seg)	11 (Pendências e comunicação) + 12 (Auditoria e compliance)
16/jun (ter)	15 (Relatórios e indicadores) + 14 Ondas 0–1 (IA: fundação + documentos)
17/jun (qua)	14 Onda 2 (IA: síntese)
Pendente	13 (Integrações) — bloqueada por terceiros · 14 Onda 3 (assistentes/RAG) — bloqueada (pgvector incompleto no SDK)

Lacunas e o que NÃO foi medido

- **Horas/dia trabalhadas** — git não registra. Inferência de alta intensidade pelo volume de commits.
- **Ferramenta/modelo de IA exato** — não há registro versionado.
- **Execução fresca da suíte completa** — não rodada aqui (exige banco + PostGIS e dois processos isolados). Contagens de teste/assert vêm de grep estático cruzado com os logs de execução no `STATE.md` (~1.554 testes / ~7.358 asserts em run completo).
- **Tempos por arquétipo (Seção 6)** — estimativas derivadas da estrutura de fases e datas de commit, não cronômetro.
- **Integrações externas (REDESIM/SEFAZ/Regin/GIS/gov.br)** — não entregues/validadas em produção (bloqueadas por terceiros); portanto fora do throughput medido.

Apêndice — Comandos de medição (auditabilidade)

```
# Janela de tempo
git log --reverse --format="%ad" --date=short | head -1      # 1º commit
git log -1 --format="%ad" --date=short                       # último commit
git log --format="%ad" --date=short | sort | uniq -c         # commits/dia
git rev-list --count HEAD                                    # total de commits
git log --format='%an' | sort | uniq -c                      # autores

# Volume
git ls-files -- 'database/migrations/*.php' | wc -l          # migrations
git ls-files -- 'app/Models/*.php' 'app/Models/**/*.php' | wc -l # models
git ls-files -- 'app/Http/Controllers/**/*.php' | wc -l     # controllers
git ls-files -- 'resources/js/pages/**/*.tsx' | wc -l        # páginas
git ls-files -- 'routes/*.php' | xargs grep -hE \
  "Route::(get|post|put|patch|delete|resource|apiResource|any|match)" # rotas
git ls-files -- 'tests/**/*.php' | xargs grep -hE \
  "public function test|#\[Test\]" | wc -l                   # métodos de teste

# Linhas (churn total vs. código real)
git log --pretty=tformat: --numstat | \
  awk '{a+=$1; r+=$2} END{print a, r}'                       # adicionadas/removidas
for d in app resources/js routes database config tests; do \
  git ls-files -- "$d" | grep -E '\.(php|ts|tsx)$' | \
  xargs wc -l | tail -1; done                                # linhas por área

# Verificações de domínio
git grep -iLE "tenant|municipio_id|landlord|stancl" -- 'app/**' 'database/**' # multi-tenant
git grep -niE "cpf" -- 'database/migrations/*.php'                  # armazenamento de CPF
```